Build a CI/CD Pipeline in Azure DevOps and connect it to SAP Cloud Foundry

Sign In to Subscribe

2021 Jan 25 9:23 AM



mchrista
Participant

22,233 Views 14 Comments 18 Likes

SAP MANAGED TAGS

DevOps

Azure DevOps is a powerful tool to organize development tasks of an any project. In addition, there is the pipeline tool available, which allows you to build continuous integration and continuous delivery pipelines. CI/CD Pipelines help you to automate and accelerate the deployment of new developed features. This blog post will give an introduction on how to use Azure DevOps to build pipelines that continuously deploy new features to SAP Cloud Foundry.

Prerequisites

To build the scenario on your own, you need trial accounts on Azure as well as on SAP Cloud Platform. Furthermore, you need any git repository. In this blog post the Git repository from GitHub is used, but any other Git repository is possible as well. In summary, you could use the following setup:

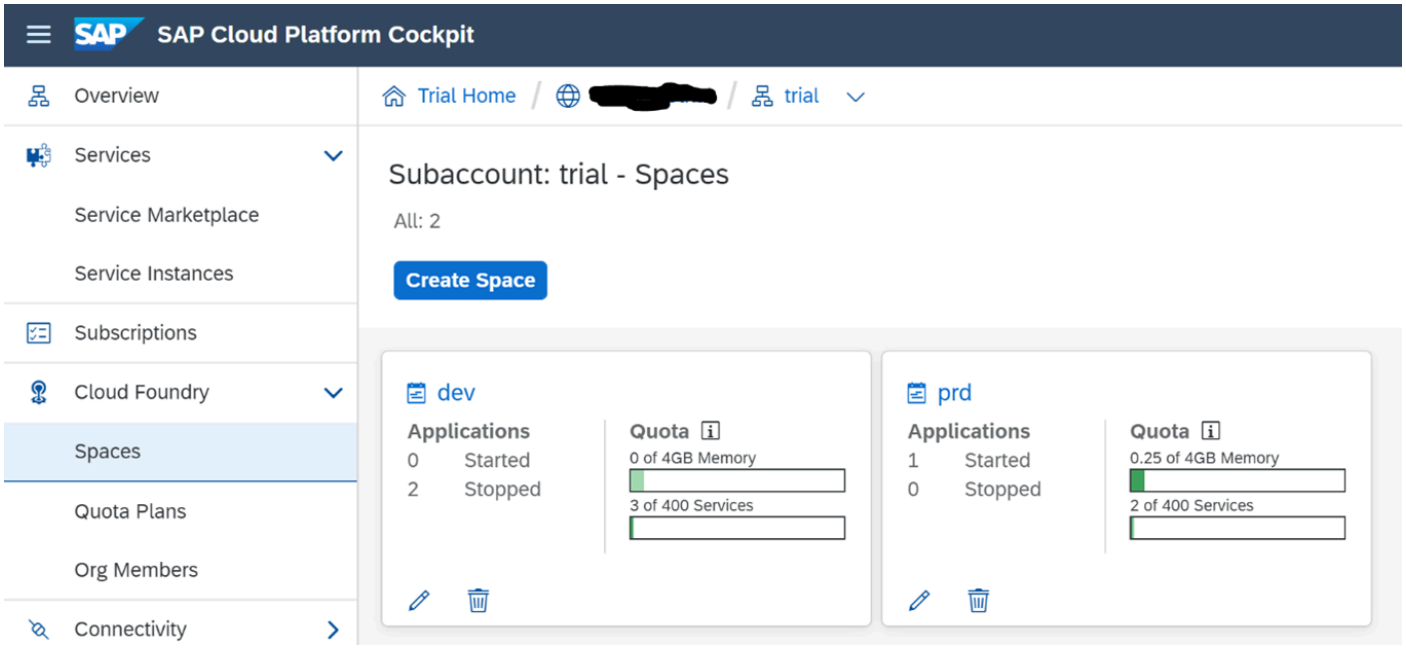
- Azure Account (Azure Pipeline Trial Account is available here)
- SAP Cloud Platform Account (Trial account is available here)
- Any Git Repository (e.g. GitHub)

Goal

We assume that your SAP application, e.g. Fiori application, is already saved in your GitHub repository. The goal is that every change to the master branch of the git repository triggers a deployment pipeline in Azure DevOps. The deployment pipeline then builds the application and deploys it after approval first to the development space and after another approval to the productive space in SAP Cloud Foundry.

Setup SAP Cloud Platform (SCP)

Within SAP Cloud Platform, we need to setup two spaces as target for our deployment. Therefore, you need to navigate to section ‘Spaces’ in the menu tree and then create two spaces. Important is to remember the names (‘dev’ and ‘prd’) of the just created spaces as they will be the targets of our deployment from Azure.



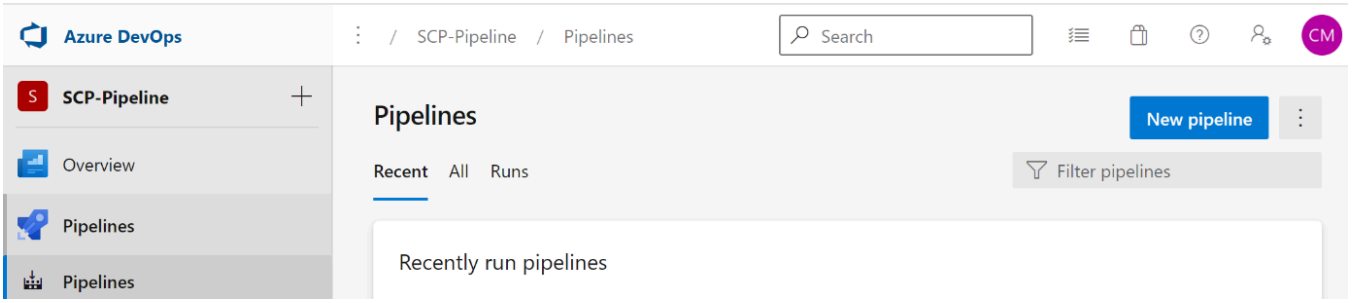
Setup Azure DevOps

After login to Azure, you need two steps to create a CI/CD pipeline. In the first step you create the build-pipeline, which is responsible to pull the application from the git repository, doing some testing and finally builds the application. In the second step you create a release pipeline which deploys the built application in the SAP Cloud Foundry spaces you have created in the previous step.

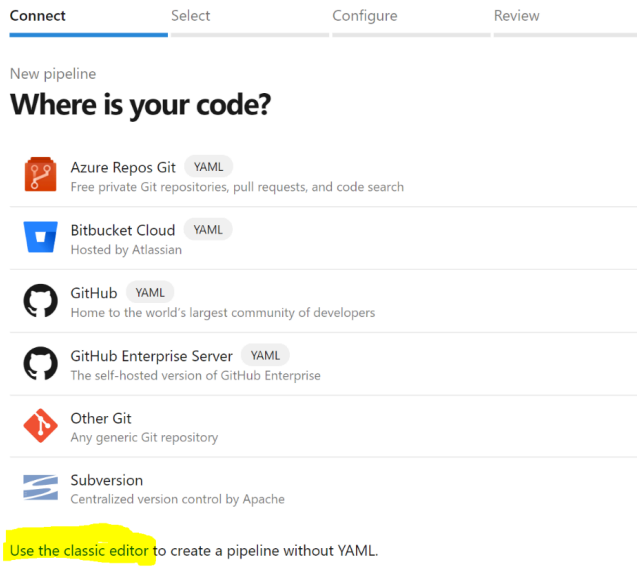
Create the Build-Pipeline

We set up a Build-Pipeline that will be triggered after every commit to our Git master branch. After triggering the pipeline pulls the application from the Git repository and builds a multi-target-application which later can be used for deployment to SAP Cloud Foundry. To configure the build-pipeline, follow the steps coming next:

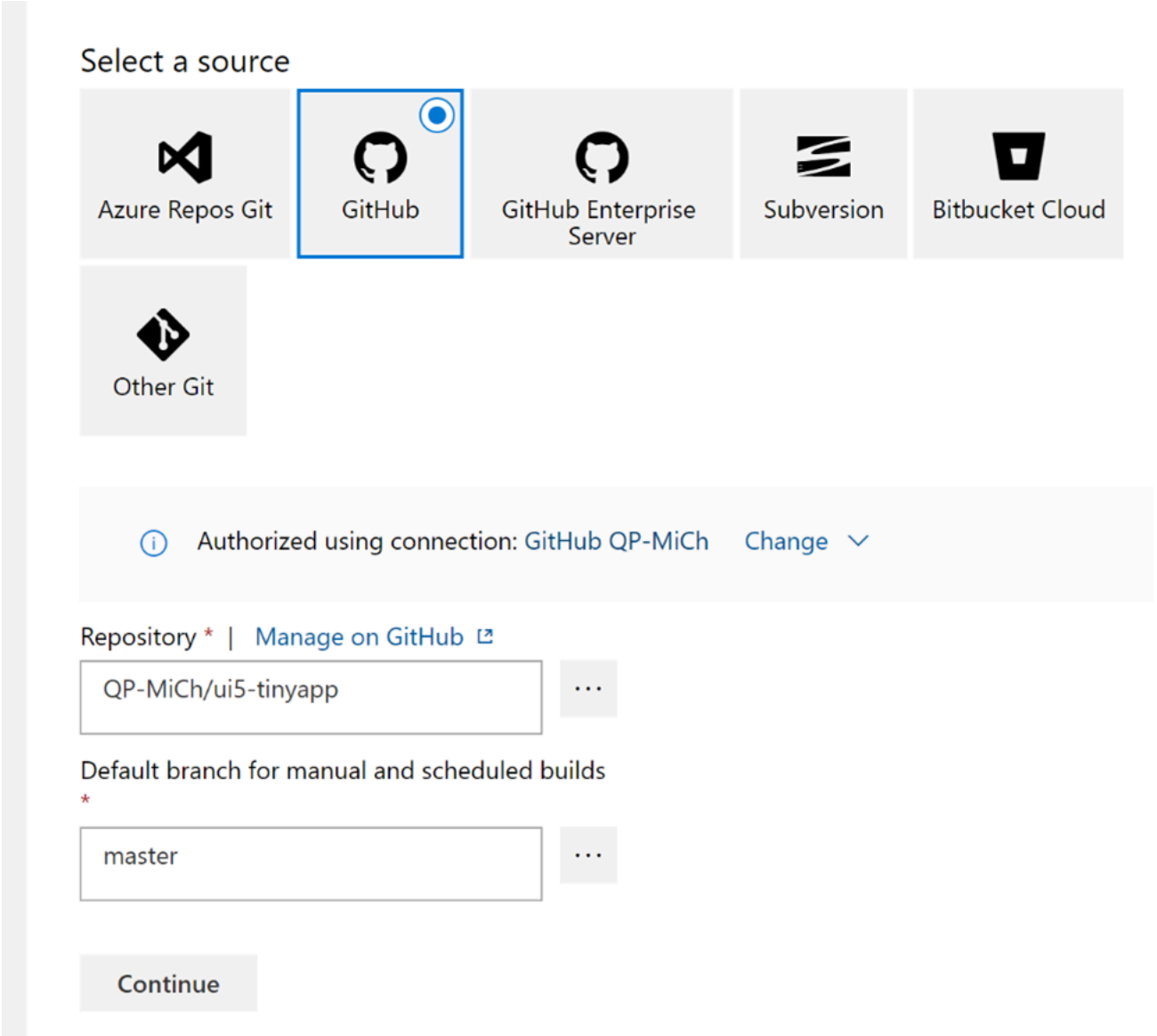
1. Select Pipelines in the menu and then use the button 'New pipeline' to create your Build-Pipeline.



2. Select "Use the classic editor" since we do not use a YAML file to configure our pipeline.



3. Now connect to your Git-Repository where your application is stored. Therefore, first login to your Git-Account and then select your repository and the branch you want to use in Azure Pipelines.



4. Choose ‘Empty Pipeline’

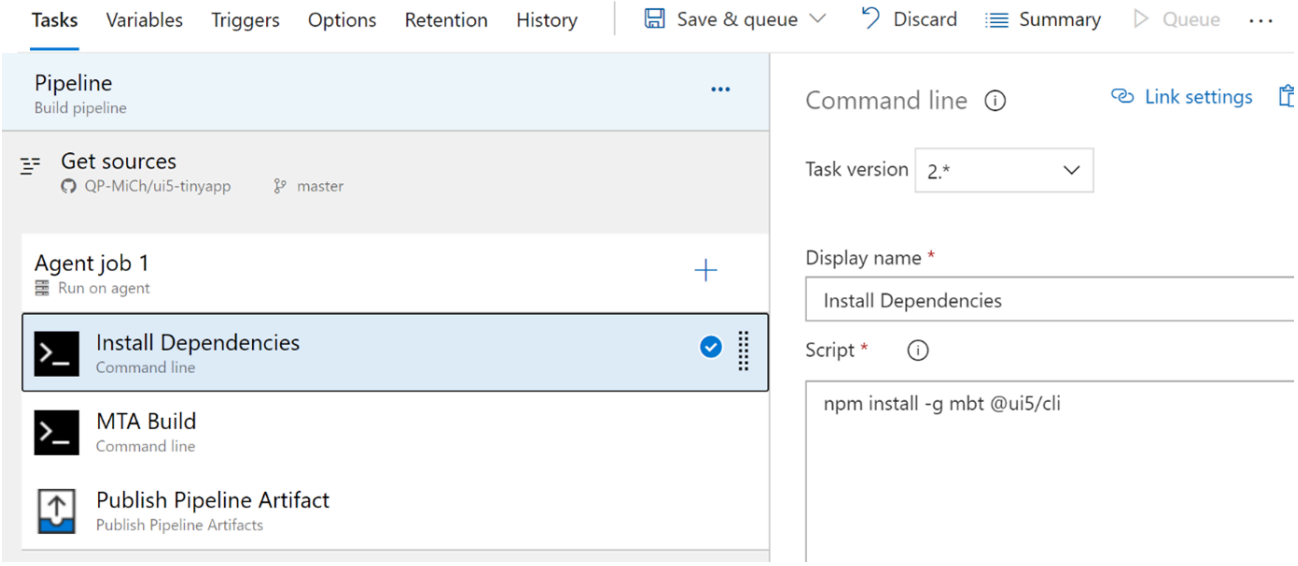


5. Create Pipeline Tasks

   In the 'Tasks' section you will now see your connected Git repository and an agent that will execute the build steps you need. For our scenario, we need three tasks to build our application and publish the built application for later deployment. A task can be added with the plus button on the right side. Perform the following three tasks:

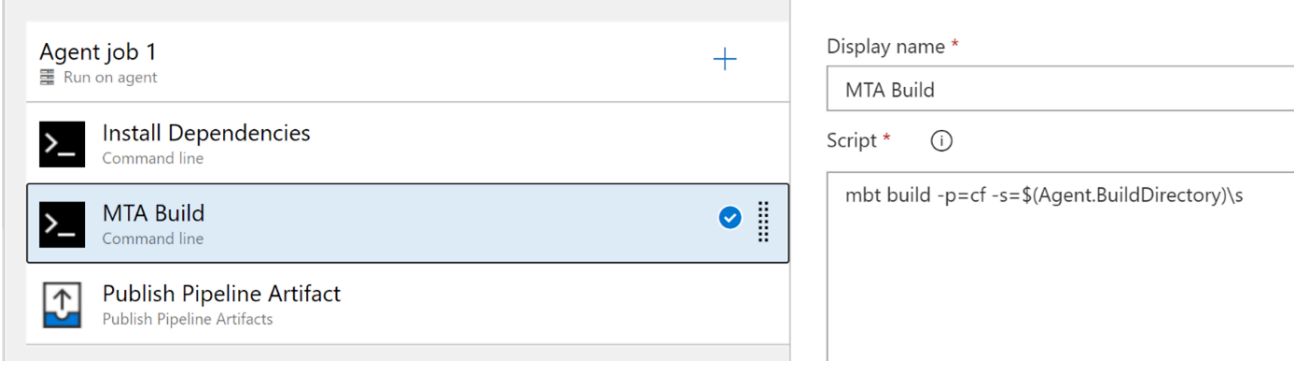
- Install Dependencies

  First add a command line task that will install dependency packages via npm. For the built, we use the mbt tool. Since it is not already known to Azure we can install it via npm. Furthermore, I used an UI5 application and therefore need to install the UI5 command line tool. If you have any other kind of application, check the logs and install the needed tools in the provided manner.

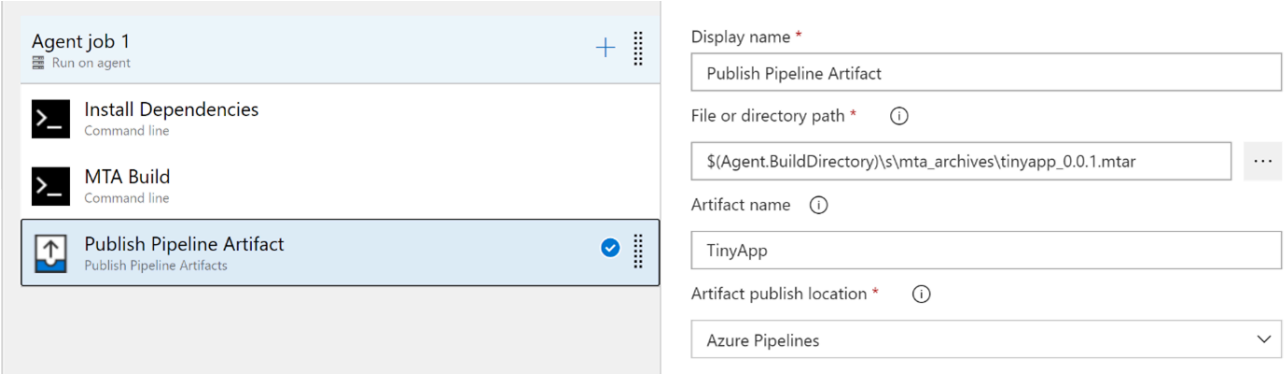


- Build the MTA

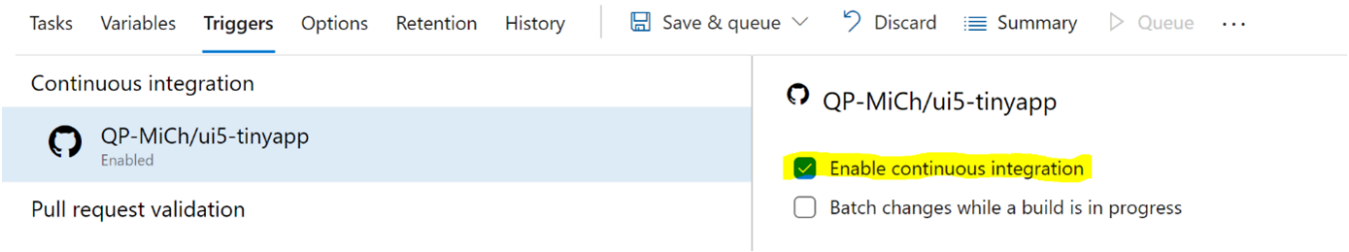
  The second task is again a command line task and builds the application. In the script section the command for building the application is shown. As target platform we set the cloud foundry environment (-p=cf) and give a target directory where the built application is saved. Here, we can use a dynamic location within Azure DevOps that is accessible in later steps as well. The built will create a folder 'mta_archives' and an mtar file within the given target directory. In my case the built application has the path '$(Agent.BuildDirectory)\s\mta_archives\tinyapp_0.0.1.mtar', where tinyapp is the name of my UI5 application.



- Publish the built application
  As a final step, we publish the created mtar file to Azure Pipelines, so that it can be used in the release pipelines later. This can be done with a so-called artifact. You can imagine an artifact like a key value store, where you give a name as a key and the file you want as a value. To do so, you need to the task type 'Publish Pipeline Artifact'. In the detail section, you give the path to your mtar-file from the previous step and give a name for the artifact.



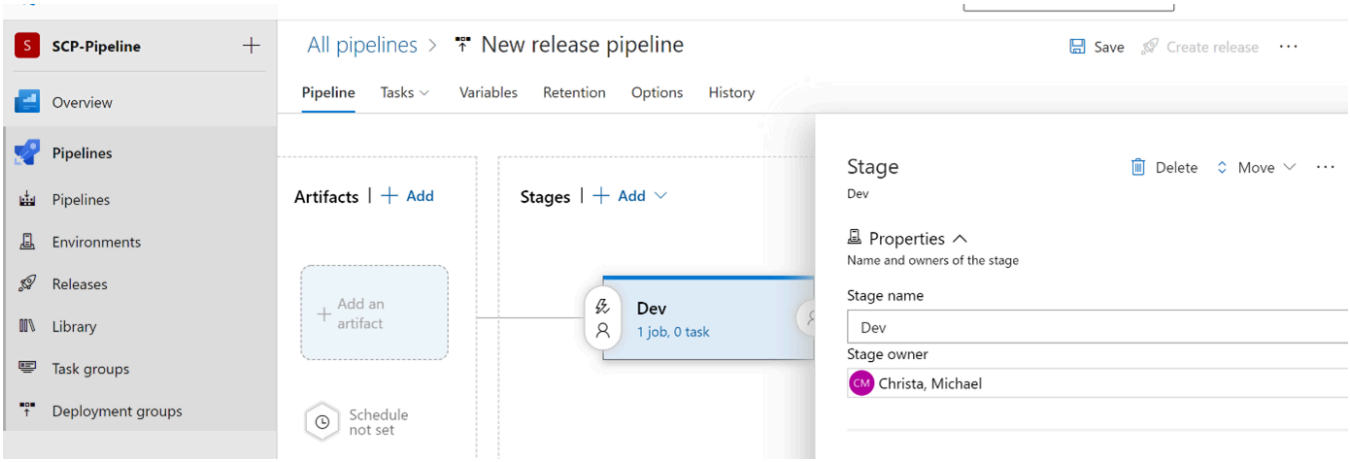
8. Configure the trigger
   After creating the tasks change to tab 'Triggers' and check the option 'Enable continuous integration'. With this setting the build pipeline is triggered on every change to the git master branch you have connected.



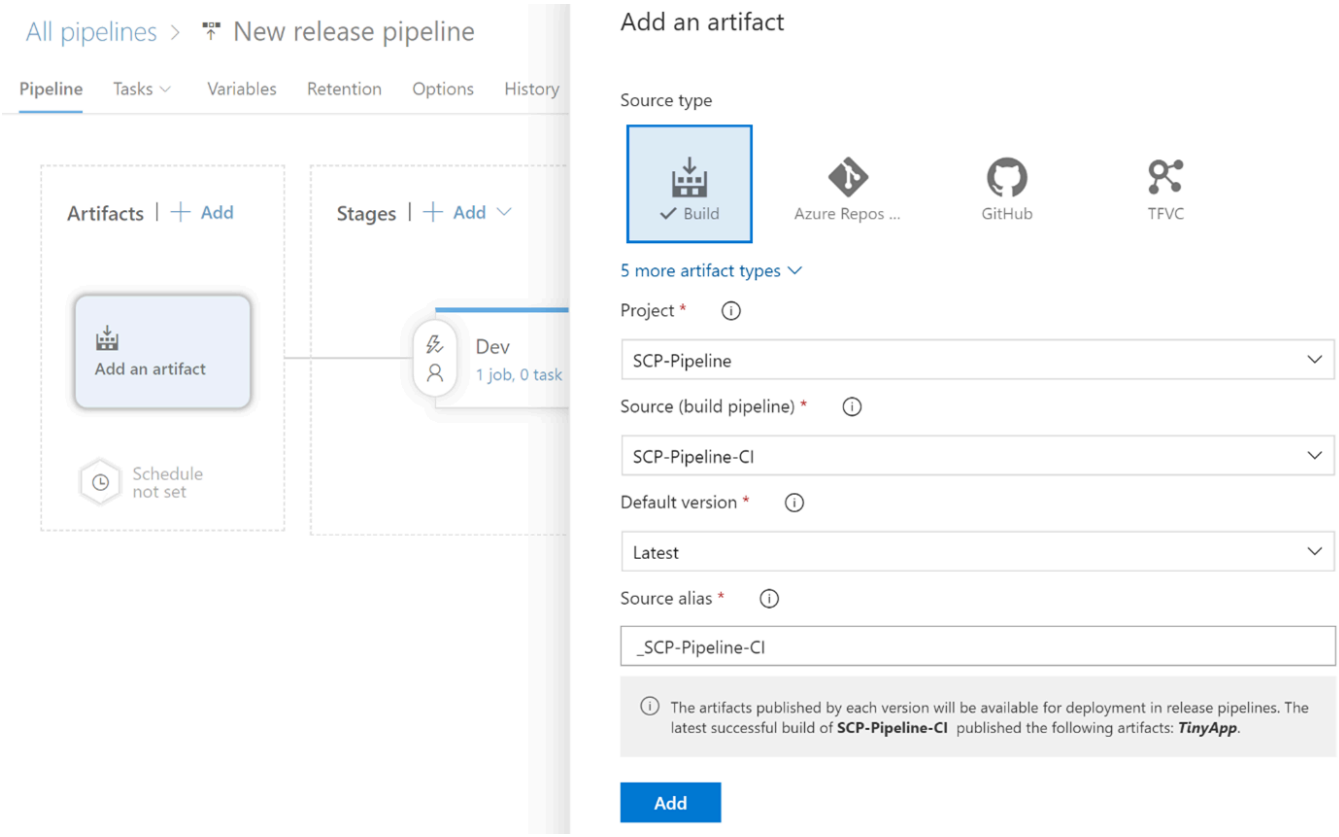
Release Pipeline

Now we can configure the deployment of the built application to SAP Cloud Foundry. This is done in a so-called release. The release for our scenario will consist of three steps. First, the application is built with the Build-Pipeline we have created. Second, after approval from a responsible person we deploy the application to the 'dev' space in SAP Cloud Foundry. Lastly, after another approval the application is deployed to the 'prd' space in SAP Cloud Foundry.

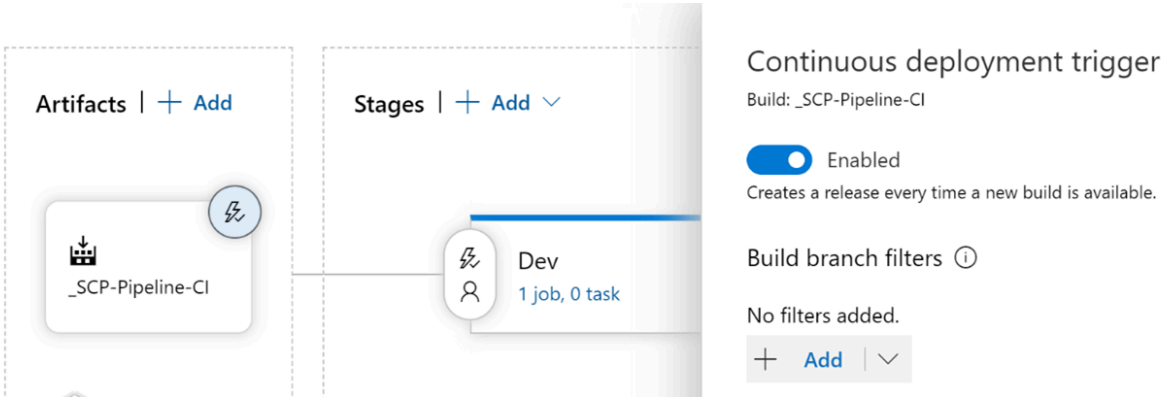
1. Create two stages for 'Dev' and 'Prd':
   In the stages area click 'Add' to create a new stage. We create two stages, 'Dev' and 'Prd', that represent the SAP Cloud Foundry spaces 'Dev' and 'Prd'. Giving the same name is not mandatory but helps to easy understand.



2. Add the Build Pipeline:
   In the artifacts area on the left-hand side, click 'Add' to create a new one. A details window will appear in which you select 'Build' as a source type and select the previous created build pipeline from the dropdown menu.

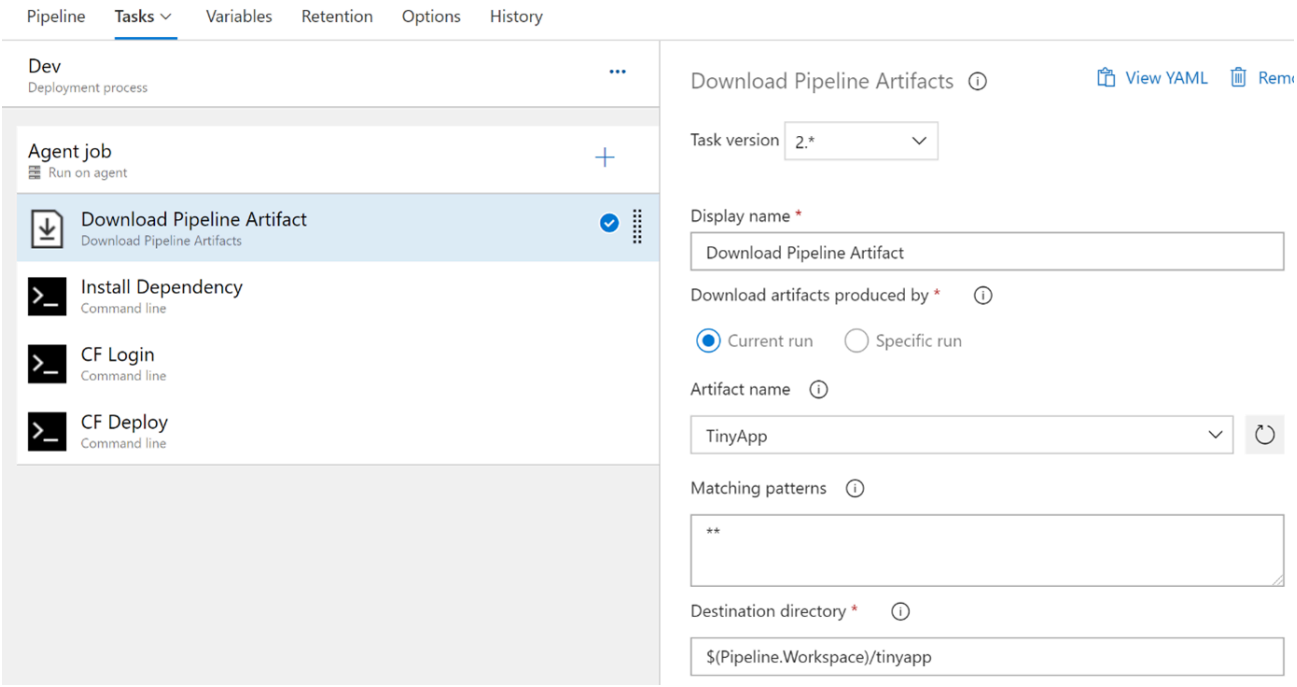


3. Configure release pipeline trigger:
   Click the flash button and enable continuous deployment trigger. With this setting, every time a new build is available the release pipeline will be triggered.

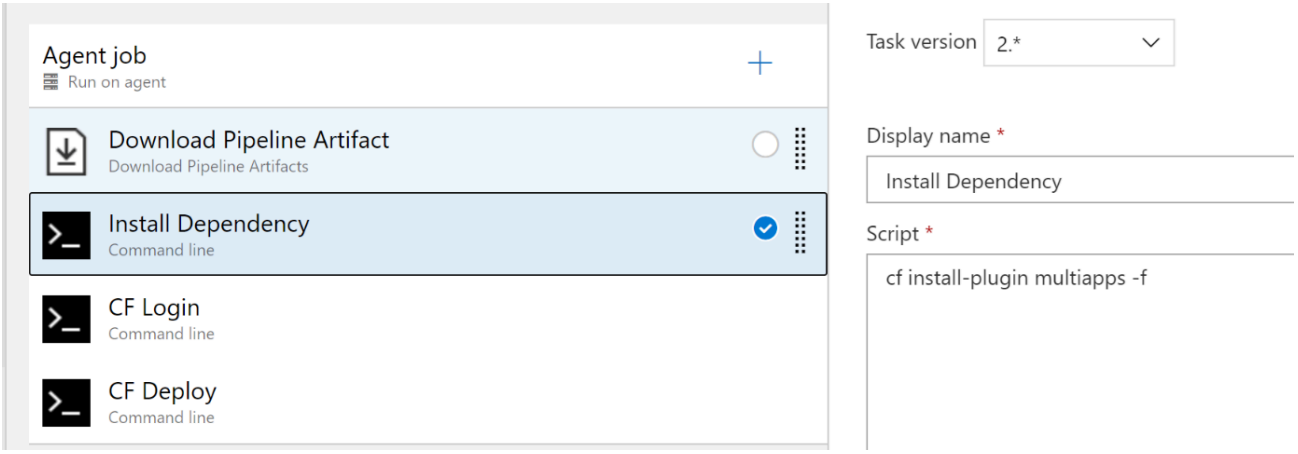


4. Configure the deployment steps:
   Back in the 'Stages' area, we need to configure the deployment tasks for 'Dev' and 'Prd'. In the presented case, the tasks are the same, so in the following only the configuration for the dev-space is shown. To configure the deployment tasks, click on the stage description '1 job, 0 taks'. Afterwards a new window will appear in which we create the following tasks:

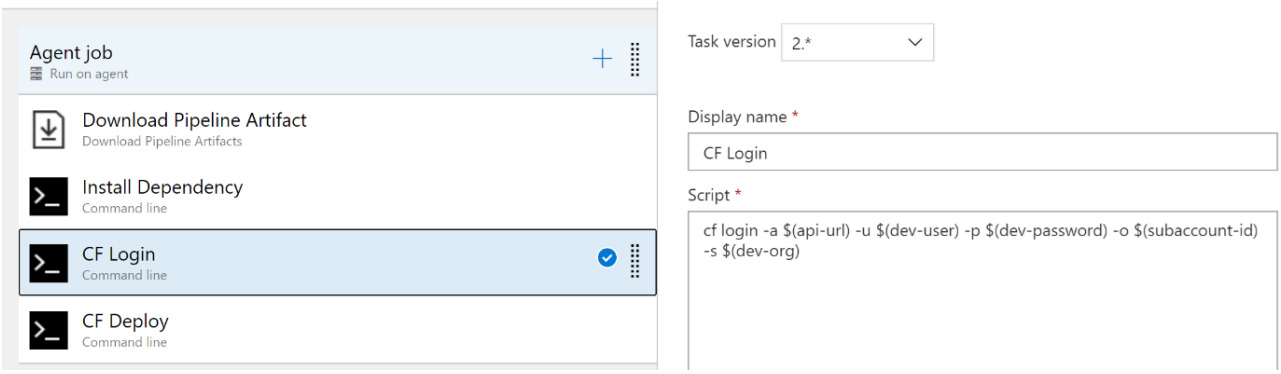
- Download the Pipeline Artifact
  In the last step of the build pipeline we uploaded the artifact 'TinyApp' that contains the deployable mtar file. In the first step of the deployment, we download the artifact and save it to the pipeline workspace.



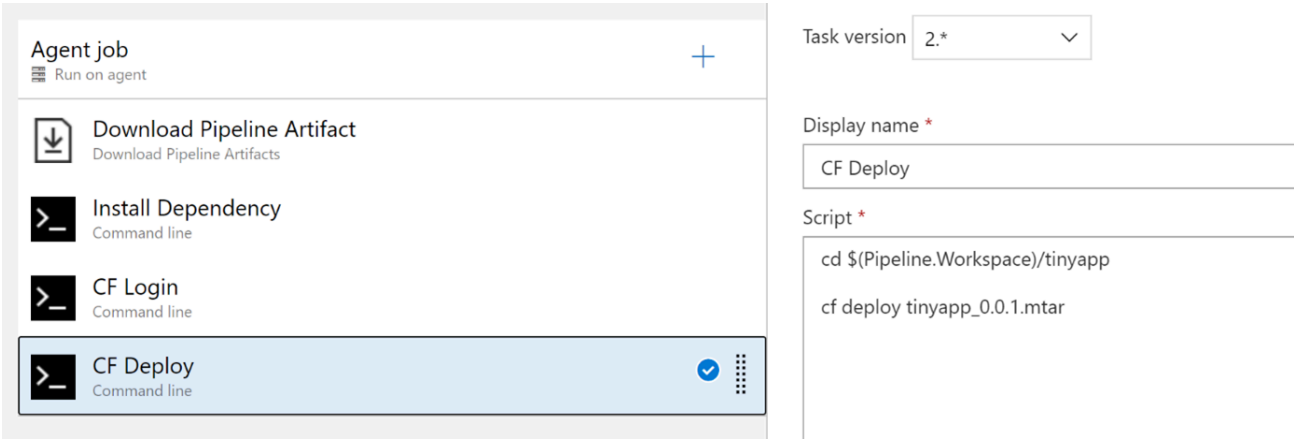
- Install dependencies
  Similarly to the build pipeline, we need to install dependencies that are required for the deployment. We need here the plugin multiapps of the cloud foundry command line interface. Further, please see the parameter -f that enables installation without any user interaction.



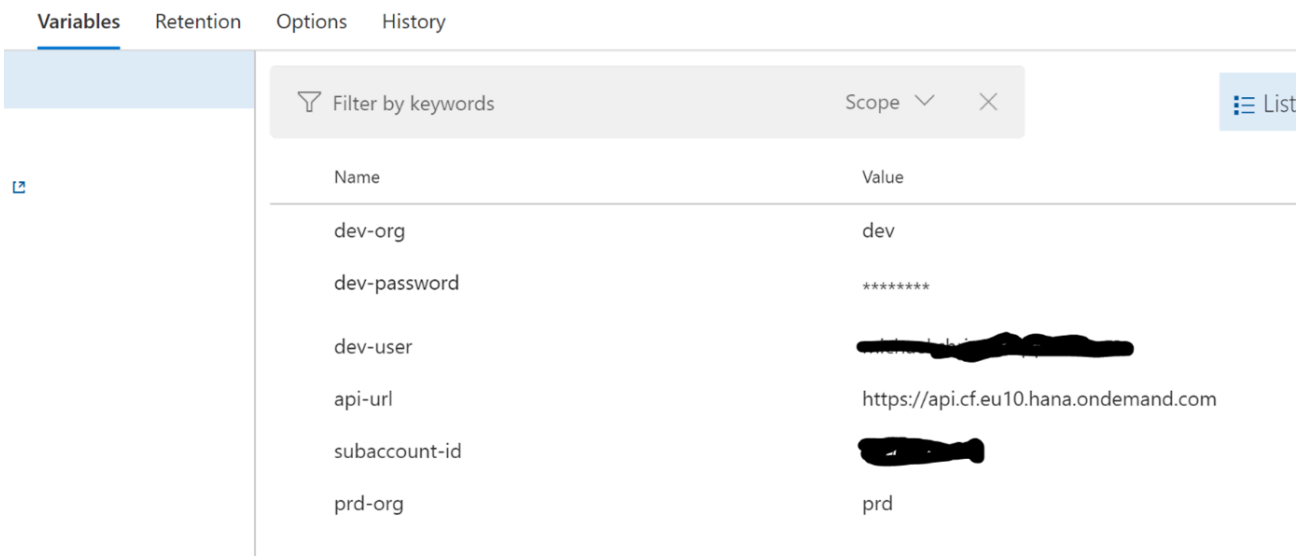
- Login to SAP Cloud Foundry
  As next step, we use the command 'cf login' to access SAP Cloud Foundry. All parameters are given as variables which we create later. Please note, that we also give the SAP Cloud Foundry space 'dev-org' here. When configuring the productive space, you need to use 'prd-org' in this command.



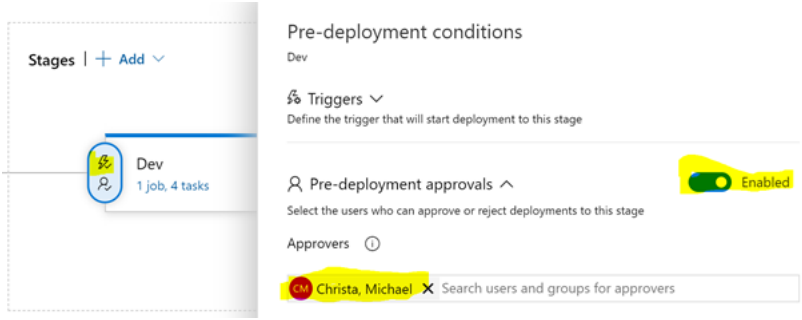
- Deploy the Application to SAP Cloud Foundry
  In the final task the deployment of the application to SAP Cloud Foundry takes place. We use the command 'cf deploy' for this purpose. The command takes the mtar file as an argument and deploys the application to the account we are currently logged in.



- Create environment variables:
  Now change to section 'Variables' and create all the variables we have used during our deployment.

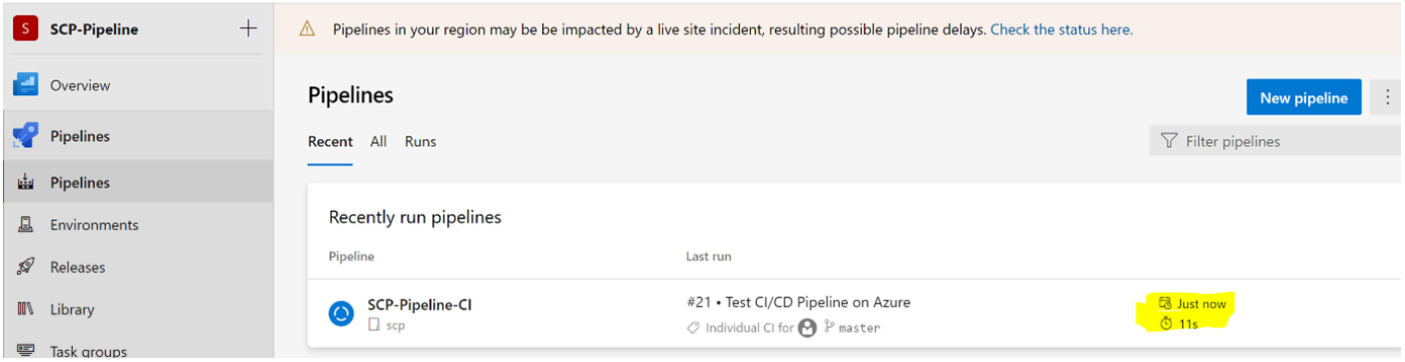


5. Enable approval:
   The final step is to enable the approval of the deployment. Therefore, click on the flash button of the stage and enable 'Pre-deployment approvals'. Furthermore, give the name of the users or the groups in the input field below.

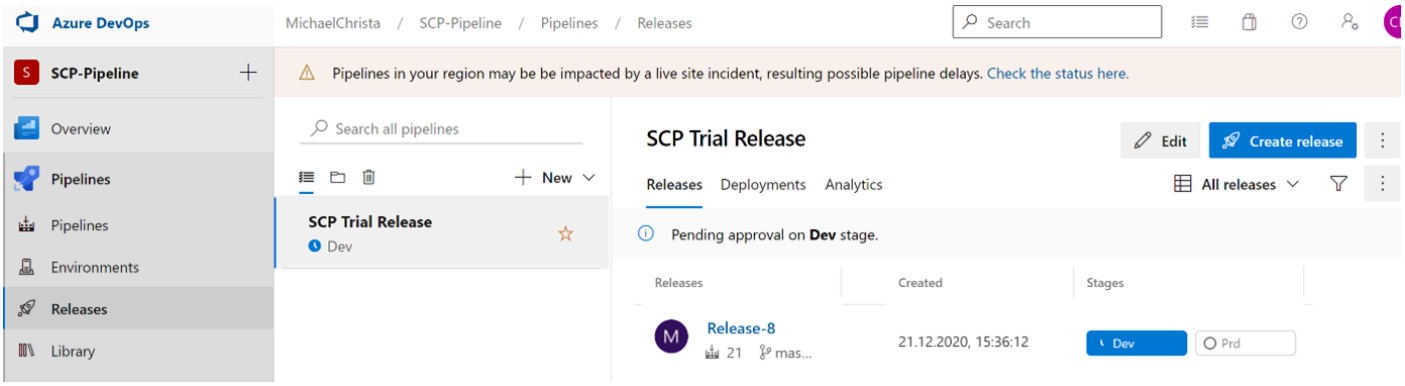


Test the Pipeline

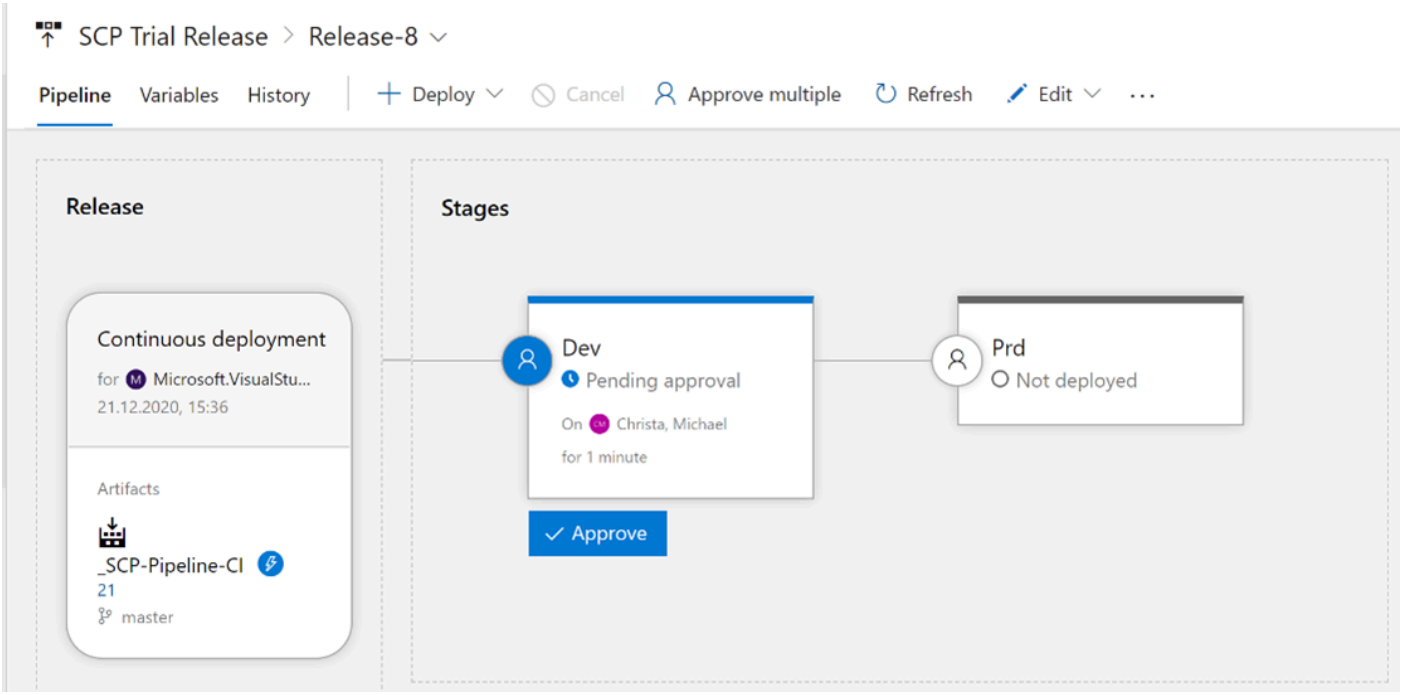
To test the configured pipeline, we just need to change our application and push those changes on the master branch of our git repository. Immediately after the commit you will see in menu item 'Pipelines' that a new build has started. The list item also displays the commit message that has triggered the build. If you click on the item, you can get detailed information about the build and can view logs.



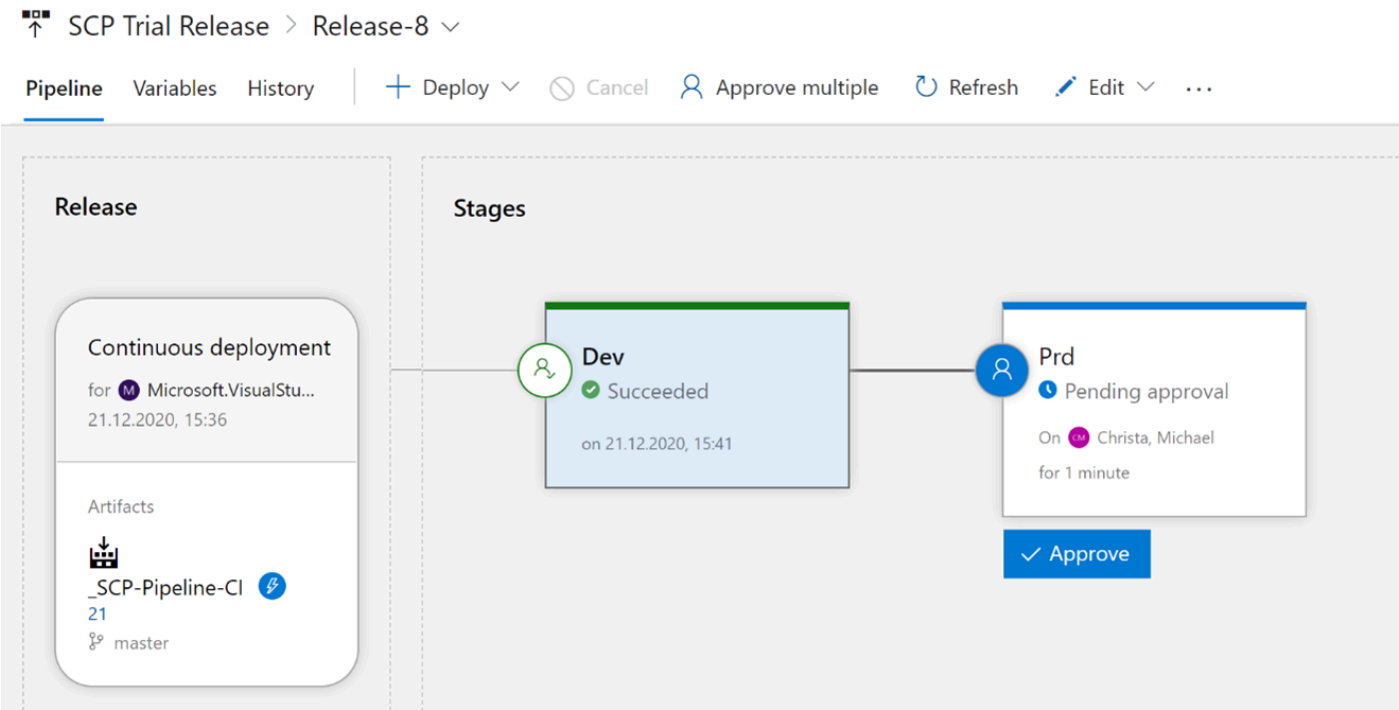
Once the build is finished, change to menu item 'Releases' and you will see that already a new release has started, since we checked the property that a new release will be started immediately after a new build has been created.



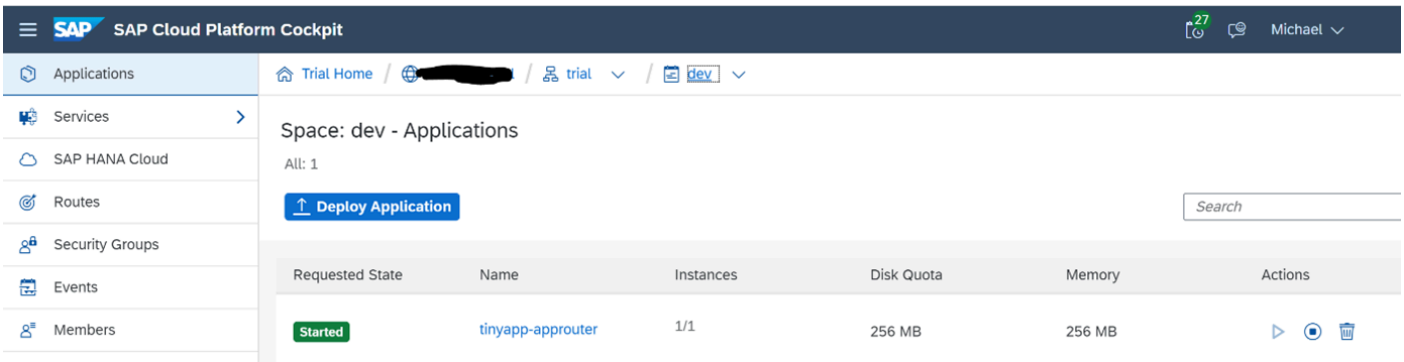
If you now click on the release item, you get to the detail page of the release and you can see that the deployment to 'Dev' requires an approval beforehand. To approve the deployment, click on the 'Approve' button. After approval the deployment is processed by Azure Pipelines. This process will take a few minutes.



Once the deployment has completed you will see the status 'Succeeded' within the dev tile and now the productive stage is waiting for approval. Here you have the option to directly approve the deployment to production or do it sometime later after you have confirmed functionality on dev stage.



To verify the deployment of your application, change back to SAP Cloud Platform and navigate to the dev space. In here, we see the just deployed application 'tinyapp' which is in state 'Started' and confirms that the deployment was successful.



Summary

In this blogs post, I gave an introduction on how Azure DevOps can be used to build a CI/CD Pipeline with SAP Cloud Platform. The given example is a very minimalistic approach that requires further tasks like automatic testing for a productive usage. However, the example is a proof of concept that integration of SAP Cloud Platform into Azure DevOps works and is configurable in few steps. Since most of the time, we do not have stand-alone SAP Cloud applications as they relate to SAP systems on-premises, my next research will be how we can integrate them into Azure DevOps. If it is also possible to integrate SAP on-premises systems to Azure DevOps, we get a powerful and flexible tool to manage our SAP application lifecycle.

About the author

Michael Christa is SAP Consultant at Q_PERIOR focusing on technology and innovation. His professional interests are developing end-to-end SAPUI5 applications as well as working with the SAP Cloud Platform.

TAGS:

azuredevops cicd pipeline

18 Likes Comment

14 Comments

vicenteveiga
Participant
2021 Feb 03 10:28 AM

Amazing Blog!

Easy to follow and worked great!

Thanks!

1 Like

TimMuchena
Participant
2021 Apr 13 5:35 PM

Excellent blog

Any idea on how to configure the deployment of the built application to SAP on-prem system?

Thanks

0 Likes

mchrista
Participant
2021 Apr 15 7:49 AM

Good question.

There is no official documentation on it. My approach would be to use project piper (https://www.project-piper.io/scenarios/upload-to-transportrequest/Readme/ since it has standardized interfaces to the abap system. Then publish the project piper system to azure devops and control it from there. Microsoft has announced at DSAG last year, that they will integrate functionalities of project piper into azure devops soon.

Another approach would be to debug the deployment process. There must be an OData-API which WebIDE as well as project piper uses. If the OData URL for deployment is known, the URL can be published to Azure DevOps, e.g. via SAP BTP API Management, and used from there.

Hope this helps to find a solution.

0 Likes

former_member625927
Discoverer
2021 Dec 06 2:36 PM

mchrista Nice post.

My question is in our pipeline we upload war artifacts to nexus which is SAP internal system. How an Azure agent can access SAP internal system or how an azure agent can create a transport request to abap system if all abap systems are SAP internal and azure dont have access to it. We currently use SAPHosted azure agent to perform these things but its not scalable to 5000 pipelines.

0 Likes



mchrista ▪▪▪
Participant
2021 Dec 07 3:44 PM

There are two questions in your comment:

1. To create transport requests in an ABAP System, I only know Project Piper. As far as I know, project piper has external APIs that can be used from Azure DevOps.

2. In order to access internal systems from Azure DevOps at all, I assume that your problem is that inbound communication is not allowed. To solve this case, Azure DevOps offers the possibility to use a self-hosted agent. The agent then connects from your internal network to Azure DevOps in the cloud but is controlled from the cloud process. Please see the following documentation: https://docs.microsoft.com/en-us/azure/devops/pipelines/agents/agents?view=azure-devops&tabs=browser

Another approach would be to use a Cloud Connector as a reverse proxy to Azure DevOps but never set this case up.

Hope this helps.

0 Likes

AshwinKatkar ▪▪▪
Participant
2022 Feb 14 12:55 PM

Hi Michael,

Nice Post..

I am getting error at build phase any thoughts on the error.

```
1 | Error: could not build the MTA project: could not execute the "make -f Makefile_20220214074611.mta p=cf mtar
```

```
2022-02-14T12:46:08.6134748Z ##[section]Starting: MTA Build
2022-02-14T12:46:08.6342443Z ==============================================================================
2022-02-14T12:46:08.6342721Z Task         : Command line
2022-02-14T12:46:08.6342959Z Description  : Run a command line script using Bash on Linux and macOS and cmd.exe on Windows
2022-02-14T12:46:08.6343181Z Version      : 2.198.0
2022-02-14T12:46:08.6343364Z Author       : Microsoft Corporation
2022-02-14T12:46:08.6343611Z Help         : https://docs.microsoft.com/azure/devops/pipelines/tasks/utility/command-line
2022-02-14T12:46:08.6343886Z ==============================================================================
2022-02-14T12:46:11.0709689Z Generating script.
2022-02-14T12:46:11.0813251Z Script contents:
2022-02-14T12:46:11.0825462Z mbt build -p=cf -s=D:\BuildAgent\_work\960\s
2022-02-14T12:46:11.1348122Z ========================== Starting Command Output ===========================
2022-02-14T12:46:11.1601572Z ##[command]"C:\Windows\system32\cmd.exe" /D /E:ON /V:OFF /S /C "CALL "D:\BuildAgent\_work\_temp\e749c198-7462-4d24-9fc1-abf591fc541d.cmd""
2022-02-14T12:46:11.3982756Z [2022-02-14 07:46:11]  INFO Cloud MTA Build Tool version 1.2.7
2022-02-14T12:46:11.3984118Z [2022-02-14 07:46:11]  INFO generating the "Makefile_20220214074611.mta" file...
2022-02-14T12:46:11.4003019Z [2022-02-14 07:46:11]  INFO done
2022-02-14T12:46:11.4003705Z [2022-02-14 07:46:11]  INFO executing the "make -f Makefile_20220214074611.mta p=cf mtar= strict=true mode=" command...
2022-02-14T12:46:11.4083259Z Error: could not build the MTA project: could not execute the "make -f Makefile_20220214074611.mta p=cf mtar= strict=true mode=" command: exec: "make":
executable file not found in %PATH%
2022-02-14T12:46:11.4742940Z ##[error]Cmd.exe exited with code '1'.
2022-02-14T12:46:11.5219424Z ##[section]Finishing: MTA Build
```

0 Likes



mchrista
Participant

2022 Feb 16 4:39 PM

Hi Ashwin,

difficult to give a qualified answer on this. Please check if the dependencies are installed without error. Second, check if the path of the execution is correct. You can put any Unix Command in the Task steps and check the output.

Best regards

Michael

0 Likes

former_member752390
Explorer

2022 Apr 27 7:00 PM

ashwin.katkar2 I am also facing similar issue ? Was your issue resolved ?

mchrista Content of my log is below

```
022-04-27T17:31:04.0112883Z Script contents: shell
2022-04-27T17:31:04.0129750Z mbt build -p=cf -s=D:\a\1\s/servicentrysheet
2022-04-27T17:31:04.0504015Z ========================== Starting Command Output ===================
2022-04-27T17:31:04.0829848Z ##[command]"C:\Windows\system32\cmd.exe" /D /E:ON /V:OFF /S /C "CALL "D
2022-04-27T17:31:04.3113888Z [2022-04-27 17:31:04]  INFO Cloud MTA Build Tool version 1.2.10
2022-04-27T17:31:04.3114650Z [2022-04-27 17:31:04]  INFO generating the "Makefile_20220427173104.mta"
2022-04-27T17:31:04.3167139Z [2022-04-27 17:31:04]  INFO done
2022-04-27T17:31:04.3168123Z [2022-04-27 17:31:04]  INFO executing the "make -f Makefile_202204271731
2022-04-27T17:31:04.7012209Z [2022-04-27 17:31:04]  INFO validating the MTA project
2022-04-27T17:31:04.8764199Z [2022-04-27 17:31:04]  INFO validating the MTA project
2022-04-27T17:31:04.9902676Z [2022-04-27 17:31:04]  INFO building the "com-prc-servicentrysheet-desti
2022-04-27T17:31:04.9903622Z [2022-04-27 17:31:04]  INFO the "com-prc-servicentrysheet-destination-co
2022-04-27T17:31:04.9904318Z [2022-04-27 17:31:04]  INFO finished building the "com-prc-servicentrysh
2022-04-27T17:31:05.1044029Z [2022-04-27 17:31:05]  INFO building the "comprcservicentrysheet" module
2022-04-27T17:31:05.1065691Z [2022-04-27 17:31:05]  INFO executing the "npm install" command...
2022-04-27T17:31:07.5740671Z npm WARN old lockfile
2022-04-27T17:31:07.5741872Z npm WARN old lockfile The package-lock.json file was created with an old
2022-04-27T17:31:07.5742766Z npm WARN old lockfile so supplemental metadata must be fetched from the
2022-04-27T17:31:07.5743424Z npm WARN old lockfile
2022-04-27T17:31:07.5744157Z npm WARN old lockfile This is a one-time fix-up, please be patient...
2022-04-27T17:31:07.5744771Z npm WARN old lockfile
2022-04-27T17:31:32.2984092Z npm WARN deprecated source-map-resolve@0.6.0: See https://github.com/lyc
2022-04-27T17:31:32.5081174Z npm WARN deprecated request-promise@4.2.6: request-promise has been depr
2022-04-27T17:31:32.7488618Z npm WARN deprecated har-validator@5.1.5: this library is no longer suppo
2022-04-27T17:31:35.9065886Z npm WARN deprecated uuid@3.4.0: Please upgrade  to version 7 or higher.
2022-04-27T17:31:36.0026366Z npm WARN deprecated request@2.88.2: request has been deprecated, see htt
2022-04-27T17:32:51.1569231Z npm ERR! code ENOTFOUND
2022-04-27T17:32:51.1570056Z npm ERR! syscall getaddrinfo
2022-04-27T17:32:51.1570349Z npm ERR! errno ENOTFOUND
2022-04-27T17:32:51.1611136Z npm ERR! network request to http://nginx-redirector.repo-cache.svc.clust
2022-04-27T17:32:51.1615755Z npm ERR! network This is a problem related to network connectivity.
2022-04-27T17:32:51.1617794Z npm ERR! network In most cases you are behind a proxy or have bad networ
2022-04-27T17:32:51.1618968Z npm ERR! network
2022-04-27T17:32:51.1619622Z npm ERR! network If you are behind a proxy, please make sure that the
2022-04-27T17:32:51.1620495Z npm ERR! network 'proxy' config is set properly.  See: 'npm help config'
2022-04-27T17:32:51.1675756Z
2022-04-27T17:32:51.1680030Z npm ERR! A complete log of this run can be found in:
2022-04-27T17:32:51.1681229Z npm ERR!     C:\npm\cache\_logs\2022-04-27T17_31_06_085Z-debug-0.log
2022-04-27T17:32:51.2146193Z ................................................................
2022-04-27T17:32:51.2233268Z make: *** [Makefile_20220427173104.mta:37: comprcservicentrysheet] Error
2022-04-27T17:32:51.2408284Z Error: could not build the MTA project: could not execute the "make -f M
2022-04-27T17:32:51.3473630Z ##[error]Cmd.exe exited with code '1'.
2022-04-27T17:32:51.3941680Z ##[section]Finishing: MTA Build
```

0 Likes

S **517083568** ...
Explorer
2022 Jun 24 7:04 AM

Dear Michael,

"Another approach would be to use a Cloud Connector as a reverse proxy to Azure DevOps but never set this case up"

I am working on this setup, SAP BTP->AzureDevops->CloudConnector->Solman(charm).

Did you get chance to setup Azure DevOps with cloud connector, if yes, would you please provide few more details and guidelines,

Thank you

Regards,

Suresh

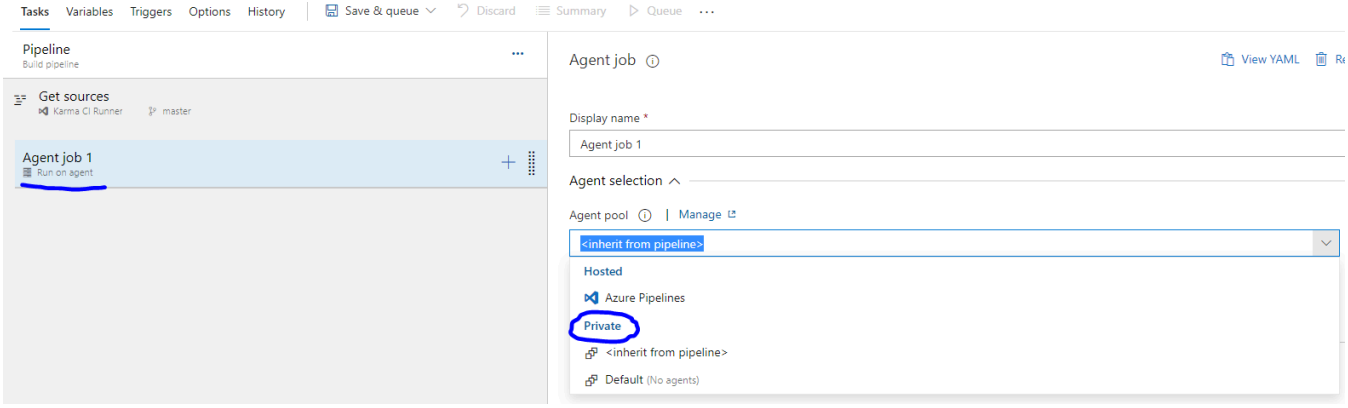
0 Likes

mchrista
Participant

2022 Jun 27 7:04 AM

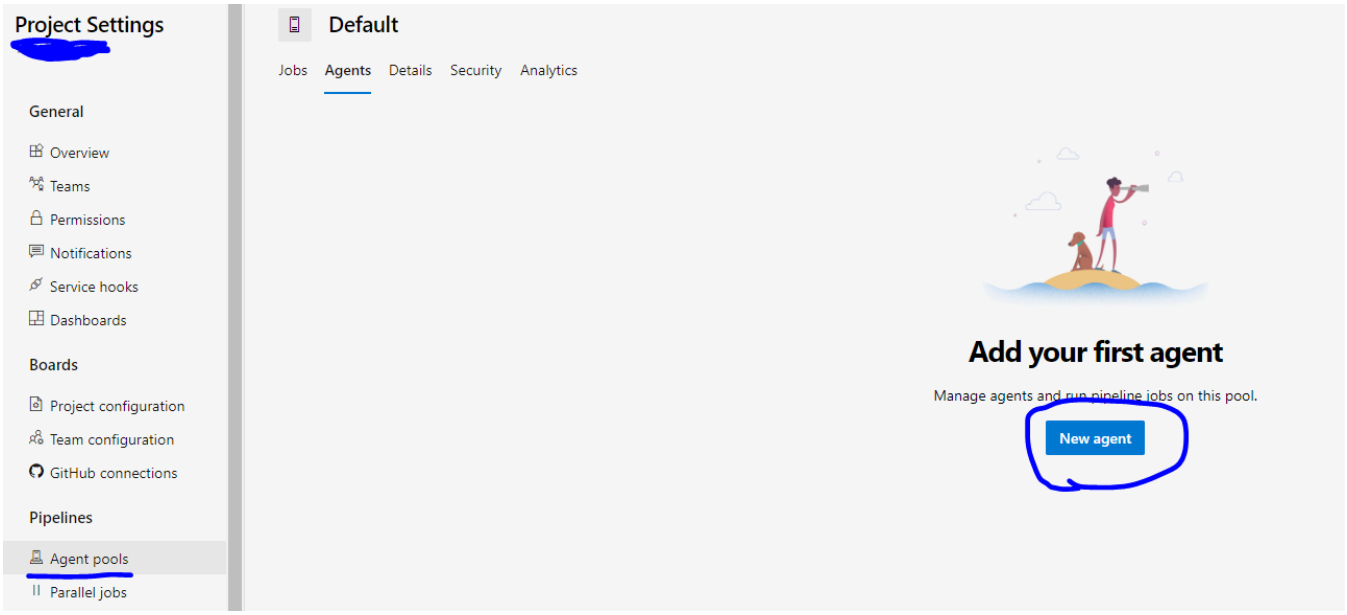
Hi Suresh,

indeed I did. I should write another blog post how to achieve this. Here some hints:
Within Azure DevOps, you can run your steps on different so called agents. This can be either a cloud system hosted by azure or you can install a agent on a system in your internal system landscape. In the following screenshot, you see on the right side, that you can select private agents:



Agent selection within Azure DevOps

In order to use a private agent, you have to set them up. Therfore, go to your project settings and click on agent-pools. Here, you will find a button to create a new agent. Once you click on it, you get a detailled instruction how to set them up on your On-Premises system.



After installation on your On-Premises system, the agent works like a reverse proxy and connects to your Azure DevOps pipeline. Since the agent is on premise, you are now also able to access solution manager.

Hope this helps.

Cheers
Michael

0 Likes



mchrista
Participant

2022 Jun 27 9:23 AM

Here is a blog, how to achieve this. The keyword is "self-hosted agent".
https://azureops.org/articles/azure-devops-self-hosted-agent/

0 Likes

former_member752390
Explorer

2022 Oct 19 3:28 PM

Hi All,

I was able to resolve this issue.

The issue is with URL path - "http://nginx-redirecto.npmrcr.repo-cache.svc.cluster.local/repository/appstudio-npm-group" appearing also in my error log.

This repository is recognizable only in SAP BAS environment and since I was trying to deploy the BAS generated code from Azure pipeline, it was giving error.

One of the solution is to replace all occurrences of this URL in file package-lock.json with below and then push your code to Git which will be further taken by your Build pipeline-

"https://registry.npmjs.com"

Regards,

Anubha Pandey

0 Likes

former_member831212
Discoverer

2022 Nov 29 10:52 AM

Hi,

can i build / deploy also SAP Analytics Cloud with this approach?

If yes, do i need to download the sources for build form SAP Analytics Cloud webfrontend, or can i host all my code in a repo for SAP Analytics Cloud?

Regards,

Robert Maier

0 Likes

thiagofcs
Explorer
2023 Apr 24 5:33 PM

Hi Everyone,

I made all the steps since the deployment part.

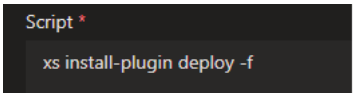
I`m doing an MTA development but using on-premise HANA XS Advanced.

I`m not sure if on the deployment I`m going to use the cf login and cf deploy or if I should use the xs login and xs deploy command.

If the xs is the case, how could I install the xs plugin at the command line in the pipeline task? The below error message is given:

```
Generating script.
Script contents: shell
xs install-plugin deploy -f
========================== Starting Command Output ========
##[command]"C:\Windows\system32\cmd.exe" /D /E:ON /V:OFF /S
'xs' is not recognized as an internal or external command,
operable program or batch file.
##[error]Cmd.exe exited with code '1'.
##[section]Finishing: Install Dependency
```

This is what I tried in the Install dependencies task:



Below the xs interface reference

XS CLI: Logon and Setup | SAP Help Portal

Appreciate any help.

Thanks,

Thiago

0 Likes

You must be a registered user to add a comment. If you've already registered, sign in. Otherwise, register and sign in.

Comment

Labels In This Area

"Job and Automation Monitoring" 1 | "RunMyJobs by Redwood" 1 | "SAP Cloud ALM" 1 | "Service Orchestration and Automation Platform" 1 | "Workload Automation" 1 | BTP 2 | DevOps 1 | Expert Insights 1 | GitHub 1 | HANA Deployment 1 | HANA Web Workbench 1 | Hybrid and Multi Cloud 1

Top Liked Authors

S	Subramani	3
	stefanre	2
M	marcelom_bovo	1
T	staerk	1

View all >

Privacy Terms of Use Copyright Legal Disclosure Trademark Support Cookie Preferences Follow